

```

from keras.layers import Input, Dense
from keras.models import Model

# This returns a tensor
inputs = Input(shape=(784,))

# a layer instance is callable on a tensor, and returns a tensor
x = Dense(64, activation='relu')(inputs)
x = Dense(64, activation='relu')(x)
predictions = Dense(10, activation='softmax')(x)

# This creates a model that includes
# the Input Layer and three Dense layers

```

Building a Functional API model in Keras

very easily. However, it has some cons such as it needs a standalone 'Input' layer at the top to specify the shape of input data.

The unique aspects of Keras functional API that differentiate it from the sequential model are:

Defining Input

Unlike the Sequential model, you must create and define a standalone Input layer that specifies the shape of input data. The input layer takes an argument that specifies the dimensionality of the input data.

Connecting Layers

The layers in the functional API model are connected pairwise by specifying where the input comes from when defining each new layer. This way of connecting layers piece by piece gives the functional API its flexibility. It is easy to start defining ad hoc graphs of layers.

Besides the two built-in Keras models – sequential and models with functional API, it also provides another class of models called Custom models. They enable you to implement custom forward-pass logic. However, they are less in use.

This chapter has described the various models in Keras – sequential and functional API models. To summarise, sequential models are highly readable, user friendly and offer high abstractions. However, they are not flexible and building complex models are difficult with them. On the contrary, models with functional APIs enables sharing of layers thus facilitating building up of complex models with ease of access. **d**